

Abstract

Penetration testing, the authorised simulation of adversarial attacks, remains inherently resistant to full automation. This stems from two structural limitations: traditional tools are constrained by signature-based detection that fails to generalise, while LLM-based autonomous agents, despite exhibiting reasoning capabilities, degrade under long-horizon and state-intensive conditions.

This thesis presents a critical analysis of automated and AI-driven approaches across four dimensions: reasoning, autonomy, adaptability, and memory management. Particular focus is given to LLM-based agents, examining paradigms such as ReAct, Chain-of-Thought, and tool-augmented reasoning, and exposing the unresolved tension between planning and execution. Existing memory mechanisms—including in-context storage and retrieval-augmented generation—are shown to be insufficient for sustained operations.

By modelling penetration testing as a long-horizon decision process under uncertainty and adversarial constraints, the study identifies fundamental systemic limitations and derives a structured research gap taxonomy. It concludes by defining the core properties required for robust next-generation autonomous systems.

Keywords: Autonomous Penetration Testing, AI Agents, LLM, Cybersecurity.

Résumé

Les tests de pénétration—la simulation autorisée et systématique d’attaques adversariales dans des conditions contrôlées—restent obstinément résistants à une automatisation exhaustive malgré des décennies d’investissements technologiques. La discipline se situe à la convergence de deux modes de défaillance distincts et non résolus : les outils d’analyse automatisés traditionnels sont architecturalement limités par des paradigmes rigides de reconnaissance de motifs; pendant ce temps, la génération émergente d’agents autonomes basés sur les LLM démontre de véritables capacités de raisonnement sémantique mais s’effondre face aux exigences cognitives imposées par des évaluations réelles et prolongées.

Cette thèse présente une analyse critique rigoureuse, axée sur la recherche, des capacités structurelles et des limitations systémiques propres à chaque grande classe d’approche automatisée et par IA pour les tests de pénétration. L’analyse est organisée autour de quatre dimensions comparatives appliquées de manière cohérente à trois générations d’automatisation. Une profondeur analytique particulière est consacrée aux agents basés sur les LLM, déconstruisant méthodiquement leurs paradigmes de raisonnement fondamentaux, dont la boucle exploratoire ReAct, la délibération par chaîne de pensée, l’exploration par arbre de pensées, et la tension structurelle persistante entre planification délibérative et exécution réactive.

Cette recherche dérive les conditions précises dans lesquelles chaque approche actuelle est structurellement insuffisante, cristallisant ces conclusions en une taxonomie de six lacunes de recherche justifiées. La thèse conclut en articulant les propriétés théoriques que les futurs systèmes autonomes doivent satisfaire pour dépasser le plafond de performance actuel en matière de sécurité offensive autonome.

Mots-clés : Tests d’intrusion autonomes, Agents IA, LLM, Cybersécurité, Lacunes de recherche.

Contents

List of Tables

List of Acronyms

AI	Artificial Intelligence
APT	Advanced Persistent Threat
CAPEC	Common Attack Pattern Enumeration and Classification
CoT	Chain-of-Thought
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
IDOR	Insecure Direct Object Reference
LLM	Large Language Model
OAST	Out-of-band Application Security Testing
OWASP	Open Web Application Security Project
RAG	Retrieval-Augmented Generation
RCE	Remote Code Execution
ReAct	Reason + Act
SAST	Static Application Security Testing
SSRF	Server-Side Request Forgery
SSTI	Server-Side Template Injection
SQLi	SQL Injection
ToT	Tree of Thoughts
TTP	Tactic, Technique, and Procedure
WAF	Web Application Firewall
XSS	Cross-Site Scripting

Chapter 1 Introduction

1.1 Scientific and Societal Context

Every networked software system represents a potential attack surface. As organisations race to modernise through continuous delivery pipelines, containerised microservices, and cloud-native deployments, the architecture of their digital estates grows in complexity faster than the security community is able to verify it. This velocity gap is not abstract risk: IBM’s 2024 Cost of a Data Breach report recorded an average breach cost of \$4.88 million USD per incident [?], while the ISC2 Cybersecurity Workforce Study documents a global shortage of four million qualified security professionals [?]. These figures are not noise; they are symptoms of a structural mismatch between the rate of software deployment and the rate of security assessment.

Against this backdrop, penetration testing has established itself as the most rigorous available mechanism for proactive, evidence-based risk assessment. A skilled human penetration tester does not merely scan; they reason. They form hypotheses about application behaviour, adapt strategy in response to observed evidence, recognise that a seemingly low-severity finding might be the pivot point for a far more serious attack chain, and ultimately determine—through actual exploitation under controlled conditions—whether a weakness can be leveraged to cause real harm. This expert reasoning is precisely what makes penetration testing valuable, and precisely what makes it expensive, slow, and impossible to scale at the pace the industry demands.

The natural response to any such mismatch between quality and scale is automation. For penetration testing, automation efforts date to the early 2000s, producing an entire industry of scanning tools, vulnerability managers, and exploit frameworks. These tools deliver genuine operational value: they are fast, consistent, and capable of cataloguing thousands of known weaknesses across large infrastructure inventories in a fraction of the time a human team would require. But they have not delivered the intelligence, the adaptability, or the semantic reasoning that give expert human assessments their value. The question of why—and what would be required to change it—is the central question this thesis investigates.

1.2 The Automation Challenge in Penetration Testing

The automation challenge in penetration testing is more structurally interesting than it might first appear. It is not simply a question of scale—doing what a human does, but faster and cheaper. It is a question of *kind*: the aspects of penetration testing that most require automation are precisely the aspects that are most resistant to it.

Consider the categories of vulnerability that cause the most severe and most costly real-world breaches. Broken access control—where a user can access another user’s data by manipulating a request parameter—is the most prevalent vulnerability in the OWASP Top 10 [?] and has featured in some of the most damaging breaches of the past decade. Detecting it requires an understanding of what a given user *should* and *should not* be permitted to access, which varies not merely between applications but between business contexts, user roles, and operational states within a single application. No finite set of pattern-matching rules can encode this understanding universally. No enumerated payload list can systematically exercise every access-control boundary in a complex application whose boundaries were never formally specified in a machine-readable form.

Business logic flaws present an equally sharp challenge. An e-commerce application that allows a user to apply a coupon code indefinitely by manipulating the order in which they perform checkout steps is exhibiting a vulnerability. The flaw is not a missing input sanitisation check; it is a failure of transactional constraint that is visible only to someone who understands what the correct business process should be. Automated tools that probe input fields with injection payloads are structurally blind to this class of flaw. Only a tester who has understood the application’s intended workflow can recognise that the actual workflow violates it.

These examples illustrate why the first generation of automated security tools—despite decades of development and enormous community investment—has not replaced expert human assessment. They set the stage for the specific question this thesis addresses: can the second generation, built on Large Language Models capable of semantic reasoning, succeed where the first generation fundamentally could not?

1.3 The LLM Promise and Its Limits

The emergence of capable Large Language Models from 2020 onward introduced a genuinely different possibility. Unlike pattern-matching scanners, LLMs are trained on vast corpora that include security research, vulnerability write-ups, exploit documentation, and adversarial reasoning. They do not merely recall memorised patterns; they can reason about novel situations, transfer understanding across domains, and generate contextually

appropriate responses to inputs they have never seen in exactly that form.

Academic research confirmed that this capability is real and measurable in security contexts. Liu et al. (2024) demonstrated that a frontier LLM, given structured descriptions of recently disclosed vulnerabilities, could autonomously identify and exploit those vulnerabilities with roughly 87% success—a performance that would be impossible for any template-based scanner starting from scratch on an unfamiliar CVE [?]. Deng et al. (2023) showed that a structured LLM agent could complete about two-thirds of a challenging set of hands-on penetration testing exercises designed for human practitioners, outperforming unaugmented model use and beginner human testers [?]. Happe and Cito (2023) demonstrated that a minimally scaffolded LLM agent could autonomously escalate privileges on most of a set of target Linux systems by reasoning through enumeration findings without any template guidance [?].

These are not marginal results. They establish a qualitative threshold: LLM-based agents can do things that pattern-based tools categorically cannot. The template ceiling—the boundary beyond which automated tools cannot generalise—has been demonstrably breached by semantic reasoning.

And yet, every research group that has built and evaluated these systems reports the same set of structural breakdowns. Tasks that require maintaining awareness of findings across many investigation steps fail at rates that are not explained by the difficulty of the task itself. Agents revisit surfaces they have already exhaustively tested because they cannot reliably determine what they have already done. They fail to pursue multi-step attack chains that their own earlier reasoning had identified as promising, because the connection between the early finding and the later opportunity has been lost in the intervening context. They cannot reliably distinguish between a hypothesis they have confirmed through evidence and a hypothesis they have fabricated through bad inference.

The pattern is clear: the models have the capability; the architectures that surround them do not provide the infrastructure needed to sustain that capability over real-world engagement durations. The limit is not LLM intelligence. The limit is agent architecture.

1.4 Research Problem Statement

This thesis takes as its central research problem the following question: *What are the precise, structurally identifiable causes of autonomous penetration testing failure, and what theoretical properties must future systems possess in order to address them?*

This framing is intentional. It asks not whether LLM-based agents can perform penetration testing at all—that question is answered affirmatively by the empirical record—but why the gap between demonstrated short-horizon capability and sustained long-horizon

performance is so large, and what kind of gap it is. The answer, as the analysis will show, is not a quantitative gap that can be closed by scaling model size or extending context windows. It is a structural gap, rooted in the absence of specific architectural properties that penetration testing categorically requires and that current agent designs do not provide.

1.5 Research Questions

The investigation proceeds from three specific research questions, ordered from the empirical to the theoretical:

- RQ1** What are the precise structural properties of traditional automated scanning approaches that prevent them from generalising beyond pre-catalogued vulnerability patterns, and why cannot those properties be overcome by engineering improvements within the same paradigm?
- RQ2** Through what specific causal mechanisms do LLM-based autonomous agent architectures—particularly their reasoning loops, tool-augmentation strategies, and memory management approaches—produce the convergent failure modes observed across empirically evaluated systems?
- RQ3** What theoretical properties—in terms of memory persistence, hypothesis lifecycle management, finding-relationship representation, and safety enforcement—are necessary conditions for an autonomous system to maintain strategic coherence across a full penetration testing engagement?

1.6 Research Hypotheses

Two hypotheses guide the analytical direction of the thesis:

H1 – The Architecture Primacy Hypothesis: The primary constraint on the performance of LLM-based penetration testing agents is not the capability of the underlying language model but the structural properties of the agent architecture surrounding it. Specifically, the inability to maintain domain-structured state external to the LLM context window is the binding constraint that prevents generalisation from short-horizon demonstrations to sustained real-world engagement.

H2 – The Convergent Failure Hypothesis: The failure modes observed across architecturally distinct LLM-based security agent systems (PentestGPT, HackingBuddyGPT, AutoAttacker) are not independent, system-specific bugs but manifestations of

a common set of structural absences that any system sharing the trajectory-as-memory model will exhibit, regardless of surface-level architectural differences.

1.7 Scope and Delimitations

This thesis is explicitly and strictly a research-driven theoretical and critical analysis. It does not present, describe, or evaluate any software implementation, experimental dataset, benchmarking sequence, or engineering artefact. Every claim made in this thesis is either supported by a citation to published research or derived by explicit logical inference from such a citation. Where the literature provides insufficient evidence to support a specific quantitative claim, this thesis does not make that claim.

The analytical scope is limited to automated web application and host-based penetration testing approaches. Network-layer and infrastructure scanning paradigms are referenced where they illuminate the comparative argument but are not the primary subjects of analysis.

1.8 Document Structure

The remainder of this thesis is organised as follows. Chapter ?? establishes the theoretical foundations required for the comparative analysis: the professional practice of penetration testing, the relevant properties of Large Language Models, and the nature of the decision-making problem that penetration testing constitutes. Chapter ?? presents the central scientific contribution of the thesis: a multi-dimensional critical comparison of three generations of automated approaches, with particular analytical depth directed at LLM-based agent reasoning paradigms, memory strategies, and the planning-versus-reaction tension. Chapter ?? synthesises the patterns identified in the comparative analysis into a causal account of why failures converge across architecturally diverse systems. Chapter ?? presents the formal research gap taxonomy, with each gap justified by evidence from the comparative analysis. Chapter ?? articulates the theoretical orientations that future approaches must explore in order to make progress on the identified gaps. Chapter ?? examines the broader implications of the analysis for cybersecurity practice, governance, and the research community. Chapter ?? synthesises all findings into a structured conclusion and a concrete research agenda.

Chapter 2 Theoretical Background

This chapter establishes the conceptual vocabulary and foundational understanding necessary to engage with the comparative analysis in Chapter ???. The exposition is deliberately focused: only concepts that are directly required for the analytical claims that follow are introduced, and each concept is introduced at the level of precision needed for those claims—not as a tutorial, not as a survey.

2.1 Penetration Testing: Practice, Standards, and Cognitive Demands

Penetration Testing

Penetration testing is the authorised, systematic simulation of adversarial attack against a target system, conducted by a skilled practitioner under controlled conditions and with the explicit objective of identifying exploitable vulnerabilities and providing concrete evidence of their real-world impact. It is distinguished from vulnerability scanning by the requirement to confirm exploitability through actual exploitation attempts, rather than probabilistic inference from detected indicators.

This definition implies something important: penetration testing is not a static analysis exercise. It is an interactive, evidence-driven investigation in which each action informs the next. The practitioner forms a hypothesis (“this endpoint may not enforce user-level access control”), tests it (by issuing a request with a different user’s identifier), interprets the result (a successful response confirms the hypothesis; a 403 or redirect refutes it), and revises their strategy accordingly. This Hypothesis–Test–Interpret cycle, repeated hundreds of times across a complex application, constitutes the cognitive core of expert penetration testing. It is a form of empirical inquiry in a hostile, partially observable environment.

The Penetration Testing Execution Standard [?] defines a seven-phase engagement life-cycle: pre-engagement (scope and authorisation), intelligence gathering, threat modelling, vulnerability identification, exploitation, post-exploitation, and reporting. Each phase places different cognitive demands on the practitioner. Intelligence gathering is predominantly a search and synthesis task. Threat modelling requires understanding the target’s business context well enough to identify what an attacker would most want to achieve.

Vulnerability identification requires both broad coverage (not missing anything testable) and contextual relevance (not testing things that cannot possibly lead to impact). The exploitation phase is the most cognitively demanding: it requires converting a theoretical weakness into a working attack under real-world defensive conditions. Post-exploitation requires reasoning about the implications of a successful breach: what further access does this enable? What data is now reachable? What persistence mechanisms can be established?

The automation challenge is sharpest at exploitation and post-exploitation. These phases require not knowledge of generic vulnerability classes, but reasoning about the *specific* instance of a vulnerability in the *specific* application under test, given the *specific* defensive infrastructure it operates behind.

2.1.1 The Vulnerability Landscape and Its Detection Paradigms

Understanding why automation founders on the exploitation phase requires examining the diversity of what “vulnerability” means across the landscape of modern web applications. The OWASP Top 10 (2021) [?] enumerates the most prevalent and impactful vulnerability classes, and a careful analysis reveals a fundamental division that is directly relevant to automation.

Some vulnerability classes are detectable primarily through pattern matching. Outdated components with known CVEs can be identified by comparing version strings against a vulnerability database. Missing security headers can be detected by checking HTTP response headers against a required list. Default credentials can be tested by trying known username-password pairs from published lists. These are tasks at which automated tools excel, and where automation provides genuine, measurable value.

Other vulnerability classes are detectable only through semantic reasoning about application intent. Broken access control (the most prevalent category, ranked A01) requires understanding who is permitted to access what, which varies per application and is never universally specifiable. Insecure design (A04) describes architecturally flawed security logic that can only be identified by someone who understands what the correct design should have been. Server-Side Request Forgery as a business-logic variant (A10 in specific deployments) requires understanding that the application’s server-side fetch capability is being exploited, which depends on recognising that the application fetches URLs at all and that those URLs are attacker-influenced. All three of these categories demand that the tester model the application’s intended behaviour and recognise that its actual behaviour departs from it in a security-relevant way. This is semantic reasoning that no template or heuristic rule can encode universally.

Table 2.1: OWASP Top 10 (2021) Classified by Detection Paradigm

Rank	Category	Description	Detection Mode
A01	Broken Access Control	Users access unauthorised resources via privilege failure	Semantic
A02	Cryptographic Failures	Sensitive data transmitted or stored without encryption	Configuration
A03	Injection	Attacker-controlled data interpreted as commands	Pattern + Semantic
A04	Insecure Design	Architecturally flawed security logic	Semantic only
A05	Security Misconfiguration	Default credentials, exposed debug interfaces, verbose errors	Pattern
A06	Outdated Components	Vulnerable third-party libraries with known CVEs	CVE database
A07	Authentication Failures	Weak session management, credential bypass logic	Pattern + Semantic
A08	Data Integrity Failures	Unsigned serialised objects, compromised update mechanisms	Semantic
A09	Logging Failures	Insufficient detection capacity (inferred from absence)	Inference
A10	SSRF	Server fetches attacker-controlled URLs	Semantic

The classification in Table ?? reveals a structural truth: the majority of the most impactful vulnerability categories require reasoning beyond pattern recognition. This is not an artefact of the particular taxonomy; it reflects the reality that the most dangerous flaws in modern software are those that emerge from the gap between intended and actual behaviour in complex, context-dependent systems—a gap that is definitionally inaccessible to systems that have no model of intended behaviour.

2.1.2 Vulnerability Chaining: The Expert Tester’s Core Skill

Beyond individual vulnerability detection, the practice that most distinguishes expert human testers from automated tools is *vulnerability chaining*: the composition of multiple individually limited findings into a high-impact exploitation path. An expert tester who discovers a Server-Side Request Forgery vulnerability does not simply record it and move

on. They reason: given that the server will fetch URLs I supply, what internal services might be reachable? Is the cloud metadata endpoint accessible? If so, are IAM credentials available through it? Those credentials may in turn enable access to object storage, further internal APIs, or account-level privilege escalation.

This chain—SSRF to metadata to credentials to storage exfiltration—crosses at least four distinct vulnerability categories, involves three different attack techniques, and requires maintaining awareness of the initial finding throughout all subsequent exploitation steps. The full exploit path has a severity that is critical even though the individual SSRF finding, considered in isolation, might be rated medium. A tool that encounters and records the SSRF in isolation, then proceeds to the next endpoint without connecting this finding to the broader exploitation opportunity, will never recover the chain.

This is precisely the kind of reasoning that current automated tools cannot sustain: not because they lack the capability to recognise SSRF as a finding, but because they cannot maintain the strategic context across many subsequent actions that connecting the chain requires.

2.2 Large Language Models: Relevant Properties for Autonomous Agents

Large Language Models are trained to predict the next token in a sequence, given all preceding tokens. Through training on sufficiently large and diverse corpora, models at scale develop capabilities that were neither explicitly trained nor anticipated: the ability to reason through multi-step problems, to recognise structural analogies across domains, to generate contextually appropriate code, and to adapt their output style and strategy to instructions provided in the input. These "emergent" capabilities are the foundation of their relevance to penetration testing.

2.2.1 Emergent Security Knowledge

The training corpora of frontier LLMs include published security research, vulnerability databases, penetration testing guides, bug bounty write-ups, and exploit documentation. Through exposure to this material at scale, the models develop a latent representation of how vulnerabilities manifest, what indicators distinguish vulnerable from non-vulnerable behaviour, and how exploitation techniques are applied in practice. This is not the same as memorising templates; it is a generalised understanding that can be applied to instances the model has never seen in exactly that form.

The practical significance of this is confirmed by Liu et al.'s (2024) result: a frontier

LLM, given a structured CVE description as context, could autonomously exploit 87% of a diverse set of one-day vulnerabilities without any template or hard-coded exploit logic [?]. Critically, the same model without the structured CVE context achieved only approximately 7% success—demonstrating that the security knowledge is present within the model but requires structured scaffolding to be reliably channelled. This distinction between the model’s intrinsic capability and its scaffolded performance is central to the analysis in Chapter ??.

2.2.2 In-Context Learning and Adaptation

Unlike traditional software, which behaves identically on identical inputs regardless of what preceded them in the session, LLMs exhibit *in-context learning*: the model’s behaviour at any given step is conditioned by the entire preceding context window. Instructions, examples, and observations provided earlier in the conversation shape how the model reasons about and responds to subsequent inputs—without any change to the model’s weights.

This property makes LLMs naturally suited to the iterative, evidence-driven structure of penetration testing: each observation can be incorporated into the model’s contextual understanding, updating its assessment of the target and shaping subsequent strategy. The limitation is that this context-dependence has a hard boundary: the context window.

2.2.3 The Context Window and the “Lost in the Middle” Problem

Every interaction with an LLM is bounded by a maximum context length, measured in tokens (roughly corresponding to word fragments). Information outside this window is completely inaccessible to the model—not degraded, not compressed, but absent. Within the window, information is not accessed with uniform reliability. Liu et al. (2023) characterised the “lost in the middle” phenomenon empirically: model performance on retrieval tasks degrades substantially when the target information is positioned in the middle of a long context, with the model showing significantly stronger recall for information at the beginning (primacy) and at the end (recency) of the input [?].

For penetration testing agents, this creates a compounding problem. As the session progresses, the context window fills with tool outputs—verbose HTTP responses, scan results, error messages—that may each be individually necessary to retain but collectively exceed the window. The model must either truncate old content (losing early findings), summarise it (losing specificity), or accept that early findings will be effectively inaccessible due to attention dilution even if they technically remain in the window. Any of these

outcomes can cause the agent to lose the thread of a multi-step investigation.

2.3 The Decision-Making Nature of Penetration Testing

To make the structural argument for why current approaches fail, it is useful to think about penetration testing as a specific type of decision-making problem. This does not require formal notation; the key properties can be characterised conceptually.

Penetration testing is a sequential decision problem in which an agent (the tester, human or automated) must select actions from a large set of possibilities, observe the results of those actions, and use those results to inform subsequent action choices. The challenge is shaped by four properties that compound each other:

High-dimensional, partially observable state: The true state of the target—which vulnerabilities it contains, how its authentication logic works, what data is accessible to which users—is not directly visible to the testing agent. The agent can only observe the target’s responses to its probes. These responses reveal partial, noisy information about the underlying state. The agent must reason from observations to inferences about state, and those inferences are always uncertain.

Sparse and deferred rewards: The vast majority of probing actions produce uninformative results. A correctly formed but non-injectable SQL query returns a normal application response. A valid but non-privileged request is handled normally. The agent receives no useful signal from these actions beyond confirmation that this particular surface does not immediately yield to this particular approach. Positive confirmation of a vulnerability is rare, and the significance of a finding may only become clear much later in the session when it can be connected to other findings.

Adversarial information suppression: The target environment does not merely fail to reveal its state; it actively suppresses informative signals. WAF systems strip error messages that would reveal backend technologies. Rate limiters make timing-based vulnerability detection ambiguous. Custom error pages conceal HTTP status codes. Authentication systems may respond identically to valid and invalid credentials to prevent enumeration. An agent operating in this environment cannot take application responses at face value; it must triangulate truth from patterns across many observations, maintaining a model of which responses might be deceptive and how.

Long-horizon dependency: The most valuable exploitation paths require connecting findings that may be separated by many actions. A reconnaissance finding from early in the session (a specific endpoint parameter handling user-supplied identifiers) may only

become relevant much later, after another finding confirms that the application's access control logic is inconsistently applied. The agent must maintain its awareness of the earlier finding throughout the intervening session, so that when the connection becomes apparent, it can be acted upon.

These four properties together define why penetration testing is genuinely difficult to automate, and why the systems that have been most effective are those that come closest to replicating the sustained, evidence-tracking hypothesis management of expert human testers.

Chapter 3 State of the Art: Critical Comparative Analysis

This chapter constitutes the core scientific contribution of this thesis. It analyses three generations of automated penetration testing approaches through a consistent comparative framework, with particular depth directed at the LLM-based agent generation that defines the current research frontier. The objective is not to describe what systems exist, but to explain—causally and with evidential support—why each approach exhibits the capabilities and limitations it does.

3.1 Comparison Framework

Four analytical dimensions are applied consistently across all approaches examined in this chapter:

- **Reasoning Capability:** Can the system infer the probable presence of a vulnerability from contextual evidence, without relying on a pre-encoded signature for that specific vulnerability type? This dimension distinguishes systems that recognise from systems that reason.
- **Autonomy Level:** To what extent does the system self-direct its action sequence throughout a session, without human intervention to resolve ambiguities, redirect strategy, or interpret ambiguous findings?
- **Adaptability:** Does the system modify its strategy in response to target-specific evidence encountered during the session? Is this adaptation rule-based (branching along pre-programmed paths) or evidence-driven (forming new hypotheses from unexpected observations)?
- **State-Memory Handling:** How does the system maintain its knowledge of what it has done, what it has found, and what it is currently investigating? How persistent, structured, and reliable is this internal model across the full duration of an engagement?

Table 3.1: Overview of Automated Penetration Testing Approaches by Generation

Generation	Reasoning	Autonomy	Adaptability	State-Memory
Signature-Based (Gen 1)	None	Moderate (static)	None	Result list only
Heuristic Active (Gen 1)	None	Moderate	Rule-based only	Site map + findings
Rule-Based Orchestration (Gen 2)	None	Moderate-high	Pre-programmed branching	Structured finding DB
LLM-Based Agents (Gen 3)	High (short sessions)	High	Evidence-driven	Volatile trajectory

3.2 First Generation: Traditional Automated Scanning

3.2.1 Signature-Based Dynamic Testing

Nuclei, developed by ProjectDiscovery and publicly released in 2020, became the de facto community standard for signature-based dynamic scanning [?]. Its architecture is straightforward in design: a YAML-based template engine encodes detection logic as a combination of a request specification (method, path, headers, body), a response matching rule (status code, header value, body substring or regex), and a severity classification. A library of over 7,000 community-maintained templates covers known CVEs, exposed administrative interfaces, default credentials, misconfigured cloud storage, and common version disclosure patterns.

The operational value of this approach is genuine and should not be dismissed. A Nuclei scan against a target with outdated, CVE-affected dependencies will reliably identify those dependencies with high precision and negligible false-positive rate, in a fraction of the time a manual review would require. For organisations seeking to ensure that their exposed software versions do not carry known, publicly exploitable weaknesses, signature-based scanning delivers measurable, reproducible value.

The structural analysis, however, exposes an inescapable boundary. Nuclei’s detection logic is entirely conditional on the prior existence of a template that encodes the specific vulnerability indicator the system is looking for. When Díaz-Verdejo et al. (2022) evaluated leading DAST tools against a corpus of intentionally vulnerable applications including

custom business logic flaws, Nuclei’s recall rate on OWASP categories requiring semantic reasoning—Broken Access Control (A01), Insecure Design (A04), and Authentication Failures in their logic-flaw variant (A07)—was negligible [?]. This is not because Nuclei lacks templates for these categories; it is because *by design, no template can encode the semantic condition for detecting them*. Broken access control is not a fixed string in a response. It is not a specific HTTP status code. It is a relationship between what the user requested, who the user is, and what they are permitted to receive—a three-way relational condition that varies per application and cannot be universally pre-specified.

The maintenance dimension compounds the problem further. The CVE database adds tens of thousands of entries annually. Nuclei template coverage of newly disclosed vulnerabilities lags disclosure by weeks to months even with an active community. The gap between known vulnerabilities and template-covered vulnerabilities is structurally growing, not shrinking.

Evaluated across the four dimensions: Reasoning capability is zero—Nuclei cannot infer; it can only recognise. Autonomy is moderate: the tool executes without intervention, but all behaviour is pre-specified. Adaptability is zero: the template set is static and does not respond to evidence gathered during the session. State-memory handling is minimal: Nuclei maintains a result log but has no model of its own investigation that would allow it to reason about what it has learned about the target.

3.2.2 Heuristic Active Scanning

OWASP ZAP [?] represents a different architectural choice that partially addresses Nuclei’s complete template dependence. Rather than matching against specific known-vulnerability patterns, ZAP implements an active scanning engine that applies broad payload families across all discovered input surfaces and analyses responses for anomalous patterns. SQL meta-characters, path traversal sequences, XSS reflection tests, and header injection strings are systematically submitted to every discovered form field, URL parameter, and cookie value, and ZAP’s heuristic engine analyses the responses for deviations that suggest vulnerability.

The theoretical advantage of this approach is that it can, in principle, detect vulnerability instances for which no specific template exists—provided the vulnerability manifests as a response anomaly that ZAP’s heuristics can recognise. A custom SQL-injectable parameter in a bespoke application might be detected through a distinctive error message or response timing anomaly even without a specific template for that application’s database backend.

The practical limitation is twofold and well-documented. First, the false-positive rate is structurally high. Because ZAP applies injection payloads uniformly without any model of

each parameter’s intended purpose, it generates anomalous responses from non-vulnerable parameters just as readily as from vulnerable ones. A search field that rejects malformed queries, a date picker that validates its input format, a text field with a maximum-length constraint—all of these produce responses that differ from what ZAP sends as input, and all of these differences look like vulnerability indicators to a context-blind heuristic engine. Díaz-Verdejo et al. found false-positive rates of 30 to 60% across evaluated applications [?], meaning that a practitioner using ZAP must manually review, investigate, and dismiss the majority of its output before any actionable findings emerge. This manual review burden does not scale.

Second, ZAP shares the flat-finding limitation with all first-generation tools: it has no model of how its findings relate to each other. Each candidate finding is independently identified and reported. The practitioner must manually reason about whether the confirmed SSRF and the confirmed misconfigured internal service are connected, and whether that connection creates an exploitation path more severe than either finding in isolation. This cross-finding reasoning is the cognitive work that first-generation tools entirely offload to the human.

Nessus and **OpenVAS** occupy a structurally similar position to Nuclei in the infrastructure domain: deep template libraries for CVE-based detection, high precision on known vulnerabilities, and near-zero coverage for anything that requires reasoning about application intent. They provide genuine value for compliance-oriented scanning and infrastructure hardening; they are not penetration testing tools in the full professional sense.

3.2.3 Domain-Specific Expert Tools: SQLMap as a Case Study

SQLMap [?] illustrates a third approach worth examining: rather than broad coverage with limited depth, it achieves near-total coverage within a precisely delimited domain. SQLMap implements six distinct SQL injection detection techniques, sophisticated WAF fingerprinting and bypass logic, and post-exploitation capabilities including automated extraction of database schemas and contents. Within its domain, it is among the most reliable and thorough automated tools available, frequently outperforming junior human testers in the specific task of confirming and exploiting SQL injection.

The analytical significance of SQLMap is what its design reveals about the scalability of deep specialisation. SQLMap requires that a human practitioner identify SQL-injectable endpoints before the tool can be applied; it cannot perform the reconnaissance phase autonomously. It cannot reason about what a confirmed SQL injection enables in the context of the broader application. If the database user has file-write privileges that would allow writing a web shell, SQLMap will not autonomously connect these two facts and

pursue the resulting Remote Code Execution opportunity unless explicitly configured to attempt it. The tool’s depth within its domain is achieved by sacrificing breadth and the inter-domain reasoning that real engagements require.

This pattern—deep competence within narrow boundaries, fundamental incapacity for the cross-domain reasoning that connects findings into chains—is a defining characteristic of the first generation as a whole.

3.2.4 Summary: Why the Template Ceiling Cannot Be Crossed from Within

The common thread across all first-generation tools is the recognition paradigm: they can identify what they have been taught to look for, and nothing else. This is not an observation about implementation quality or community effort. It is a logical consequence of the paradigm. Any detection system that operates by pattern matching against a known-vulnerability model is bounded by the completeness of that model. Novel vulnerabilities, context-specific business logic flaws, and multi-step exploitation chains that cross vulnerability categories are definitionally invisible to recognition-based systems.

The Template Ceiling

The template ceiling is not a resource problem. No amount of additional template-writing, engineering effort, or community contribution can enable a recognition-based system to detect a vulnerability class whose detection requires understanding what the application *is supposed to do*. This understanding is semantic, contextual, and application-specific. It is the foundation of expert penetration testing, and it cannot be encoded in a finite set of patterns.

3.3 Second Generation: Rule-Based Automation and Expert Systems

The second generation of automated security tools attempted to address the coordination and sequencing limitations of standalone first-generation scanners without abandoning the deterministic, rule-based foundation that makes their behaviour predictable and auditable. Orchestration frameworks and integrated exploitation suites—including the automation layers of platforms such as Metasploit [?]¹—applied conditional branching logic to chain tool invocations: a confirmed open port triggers a protocol-specific enumeration module, which may trigger a vulnerability-specific exploitation module upon confirming a matching version.

This advance is real but limited. Conditional branching introduces a primitive form of adaptability: the tool sequence is not fully fixed in advance but depends on what previous steps reveal. This is strictly better than the completely static behaviour of first-generation tools. However, the adaptability is bounded by the *coverage completeness assumption*: the system can only branch along paths that a human developer anticipated, defined, and programmed. Any environmental deviation from the anticipated conditions—an unexpected middleware layer, a non-standard authentication scheme, a combination of findings that creates a novel attack path not represented in the ruleset—leaves the system without a branch to follow. The ruleset exhausts itself, and the system stops adapting.

The second generation also brought real improvements in finding management: integrated tools maintain structured result databases that persist across the session and are queryable by later modules. This is an improvement over the flat result logs of first-generation tools. But the database is used for condition evaluation in subsequent rules, not for strategic inference. The system cannot ask “given all my confirmed findings taken together, what is the most valuable next step?” That question requires reasoning, not rule lookup.

The second-generation limitation, in summary, is the same as the first generation’s: the absence of a mechanism to reason about novel situations. The ruleset can be arbitrarily large; it cannot be infinite. And real-world targets, in all their specific, contextual, business-logic-laden complexity, routinely present situations outside any finite ruleset.

3.4 Third Generation: AI-Driven Autonomous Agents

The emergence of frontier Large Language Models created the conditions for a qualitatively different approach. Rather than encoding security knowledge as templates or rules—both of which are finite and require prior specification—an LLM encodes security knowledge as a generalised representation that can be applied to novel instances. The agent does not need a template for the specific vulnerability in front of it; it needs to understand security well enough to reason about what the instance means.

3.4.1 Demonstrated Capabilities: Establishing the Baseline

Before analysing the limitations of LLM-based agents, it is necessary to establish what they can do—both because the capabilities are genuine and significant, and because the limitations are only analytically meaningful against an accurate picture of the capability baseline.

CVE-Level Exploitation. Liu et al. (2024) provided the most direct measurement of LLM security reasoning capability [?]. When a frontier LLM was provided with structured

CVE descriptions and tasked with autonomously exploiting the described vulnerabilities across a diverse set of real software, it achieved approximately 87% success. The same model without the structured CVE context achieved approximately 7%. This 12.4-fold performance gap, attributable solely to the removal of structured external context, is a pivotal result: it demonstrates both that the model’s security knowledge is sufficient for exploitation reasoning, and that structured contextual scaffolding is necessary to channel that knowledge reliably.

Hands-On Penetration Testing Tasks. Deng et al. (2023) evaluated PentestGPT against HackTheBox challenges—hands-on, flag-based penetration testing exercises designed for human practitioners [?]. The structured agent achieved approximately 67% task completion, compared to 14.3% for an unaugmented LLM baseline and approximately 40% for beginner human practitioners. The improvement attributable to the agent’s architectural scaffolding (task-specific module separation) over the raw model demonstrates that architecture substantially shapes how effectively the model’s intrinsic capability is channelled.

General Host Security Reasoning. Happe and Cito (2023) evaluated LLM-based privilege escalation on a set of target Linux hosts with minimal agent scaffolding [?]. Without explicit guidance on which techniques to apply, the LLM agent enumerated available binaries, identified SUID exploitation opportunities, investigated writable configuration files, and escalated privileges on the majority of target hosts. This demonstrates that the model’s security reasoning generalises beyond previously described vulnerabilities—it can reason from first-principles security knowledge applied to what the current target reveals about itself.

These results collectively establish that LLMs have crossed the template ceiling. They can reason about novel vulnerability instances without pattern-matching templates. They can chain findings conceptually, at least within a single reasoning step. They can adapt their strategy based on observed evidence. The question that the remainder of this chapter analyses is why these capabilities, demonstrably present, fail to produce reliable sustained performance across full engagements.

3.4.2 LLM-Based Agent Reasoning Paradigms: A Deep Analysis

The way an LLM’s reasoning is structured and elicited by the surrounding agent architecture is not a secondary concern. It is the primary determinant of whether the model’s intrinsic capability can be sustained across a multi-hour, multi-hundred-action security investigation. This section analyses the principal reasoning paradigms deployed in LLM-based security agents, examining each for both its demonstrated capabilities and its structural limitations in sustained penetration testing contexts.

The ReAct Paradigm: Interleaved Reasoning and Action

The Reason-Act (ReAct) paradigm, introduced by Yao et al. (2022), is the architectural foundation of virtually every LLM-based autonomous agent currently in the literature, including all evaluated security agents [?]. Its central design principle is the interleaving of explicit textual reasoning traces and tool invocations within a single continuous generation loop.

At each step, the agent receives the full accumulated conversation history as input. It first generates a textual reasoning trace—sometimes called a “thought”—that articulates its interpretation of the current evidence and what action it intends to take and why. It then generates an action specification: which tool to call and with what parameters. The tool is executed, its output is appended to the conversation as an observation, and the cycle repeats. The agent’s strategy evolves step by step, with each reasoning trace shaped by all preceding observations.

Why ReAct advances the state of the art: ReAct’s interleaving design enables a form of real-time hypothesis revision that neither pure planning (which commits to a plan before taking any action) nor pure reaction (which takes actions without explicit reasoning) provides. When an observation at step t contradicts the expectation that motivated the action at step $t - 1$, the reasoning trace at step $t + 1$ can explicitly acknowledge the contradiction and revise the strategy. Yao et al. demonstrated that this mid-task course correction produces substantially better performance than chain-of-thought reasoning alone or action execution alone on multi-step tasks requiring information gathering [?]. For penetration testing—where almost every probe returns unexpected information that should update the tester’s model of the target—this adaptability is theoretically well-matched to the task structure.

The structural failure of ReAct in sustained security contexts: The fundamental problem with ReAct as applied to penetration testing is that the agent’s *sole persistent memory of its entire session* is the conversation history accumulating in its context window. Every reasoning trace, every action specification, every tool observation is appended to this linear text buffer. As the session extends, the buffer grows. After several hundred iterations of a real engagement—a modest session for a complex web application—the buffer may contain tens of thousands of tokens of HTTP responses, scan output, and error messages.

Three distinct failure modes emerge from this design:

First, *context window overflow*: when the accumulated history exceeds the model’s context limit, some portion of early history must be truncated or compressed. Any compression operation is inherently lossy. Confirmed findings from early in the session,

rejected hypotheses that should exclude certain approaches from further consideration, and strategic reasoning about multi-step chains—all of these are at risk of being absent or distorted at the point in the session when they are most needed.

Second, *attention dilution within the window*: Liu et al. (2023) documented that even information that technically remains within the context window is not recalled with uniform reliability [?]. The transformer’s self-attention mechanism exhibits primacy and recency bias; information positioned in the middle of a long context is under-attended relative to what precedes and follows it. A confirmed vulnerability finding from 80 steps ago may be literally present in the context but effectively inaccessible to the model’s generation at step 160—not because it was truncated but because the attention mechanism has moved on.

Third, *action space inconsistency*: because the agent has no structured record of what it has tested, it cannot determine from conversation history alone whether it has already tried a specific payload against a specific endpoint. The history *contains* this information, but it is embedded in natural language that may be phrased differently each time the model references it, and it may be diluted by hundreds of subsequent actions. The result is that the agent will regularly re-test surfaces it has already exhausted—the “thrashing” behaviour documented across empirical evaluations of PentestGPT and HackingBuddyGPT—not because it has made a strategic decision to retry, but because it has lost reliable access to the record of its own prior actions.

ReAct in Security: Capability and Structural Limit

ReAct enables genuine real-time hypothesis revision grounded in observed evidence—a qualitative advance over both pure planning and pure pattern matching. Its structural limit is that the trajectory-as-memory model scales linearly with session length while the context window is fixed, making sustained coherence across real-world engagement durations structurally unstable. This is not an LLM capability problem. It is an architectural problem with the ReAct paradigm as applied to long-horizon tasks.

Chain-of-Thought: Deliberative Planning and Its Failures

Chain-of-Thought (CoT) prompting, characterised by Wei et al. (2022) [?], elicits explicit multi-step verbal reasoning from LLMs by instructing them to articulate intermediate reasoning steps before producing a final response. Unlike ReAct’s step-by-step interleaving, standard CoT generates a complete reasoning chain before any action is taken. It is therefore a planning paradigm: the agent reasons about what it should do, commits to a plan, and then executes it.

The security reasoning advantage of CoT is measurable on isolated, closed-world tasks.

When an LLM is presented with a complete application description and asked to reason systematically about its potential vulnerabilities, CoT-style prompting produces more thorough and structurally coherent analysis than unstructured generation. The explicit step-by-step articulation prevents the model from reaching conclusions prematurely, forces decomposition of complex reasoning into verifiable sub-steps, and produces outputs that are more easily scrutinised for logical errors.

The limitation of CoT in operational security contexts is the *open-world planning failure*. CoT planning assumes that the world state at the time of planning will persist, at least approximately, through plan execution. In penetration testing, this assumption is systematically violated. A plan generated on the basis of initial reconnaissance may become obsolete the moment the first active probe reveals information that the reconnaissance did not anticipate. A WAF that blocks certain payloads, a rate limiter that collapses timing-based inference, an authentication mechanism that presents different behaviour to automated requests—any of these can invalidate a CoT plan at its first execution step, leaving the agent with a sequence of instructions that are no longer applicable and no principled mechanism for generating a replacement plan from the new evidence.

Attempts to combine CoT planning with ReAct execution—generating a high-level plan via CoT, then executing individual steps via interactive ReAct loops—introduce an additional complication: *plan-reality divergence*. When early execution results contradict the CoT plan’s assumptions, the agent must either follow the plan (producing strategically irrational behaviour that ignores contradicting evidence) or abandon it (wasting the planning computation and potentially creating confusion about which instructions are now operative). Neither resolution is satisfactory, and the published literature on security agents does not demonstrate a principled approach to this divergence.

Tree of Thoughts: Exploratory Deliberation

Tree of Thoughts (ToT), introduced by Yao et al. (2023) [?], extends the CoT paradigm from a linear reasoning chain to a tree-structured exploration of alternative reasoning paths. The model generates multiple candidate next-reasoning-steps from any given state, evaluates the relative promise of each (often using a second LLM call as an evaluator), selects the most promising branch for further exploration, and continues the process recursively. This resembles Monte Carlo Tree Search—a principled exploration algorithm—applied to reasoning trajectories rather than game states.

The conceptual capability that ToT offers security agents is exactly what CoT lacks: the ability to hold multiple competing hypotheses simultaneously, evaluate their relative promise without committing, and maintain fallback options if the most-promising branch turns out to be a dead end. In security terms, this corresponds to parallel reasoning about

multiple potential exploitation paths from the current evidence base, with the flexibility to switch between them as new evidence arrives.

The practical limitations of ToT in sustained security contexts are severe. First, tree expansion is computationally expensive: each branch requires at least one additional LLM call for generation and one for evaluation, with costs multiplying at each level of the tree. For short-horizon reasoning problems—selecting the best approach to a single known vulnerability—this overhead is acceptable. For full penetration testing sessions requiring hundreds of consecutive decisions, it is prohibitive with current API economics.

Second, the utility evaluator—the model that assesses how promising each branch is—suffers from the same long-context limitations as the primary reasoning model. A utility evaluator assessing the promise of a reasoning branch that references findings from 100 steps ago is operating from a degraded information base; its evaluations become unreliable precisely where the branching decision is most consequential (late in the session, when many branches have been explored and the remaining opportunities are subtle).

Third, the tree structure itself is a form of in-context state that grows with session depth, compounding rather than resolving the trajectory explosion problem identified in the ReAct analysis. Published applications of ToT to security tasks have therefore been limited to short-horizon problems where these costs are tractable [?]. Extension to full-engagement security assessment remains an open theoretical problem.

Tool-Augmented Inference: Capabilities and Their Costs

Tool augmentation—equipping an LLM agent with the ability to invoke external functions, APIs, or system commands and receive their outputs—is what elevates security reasoning from speculation to grounded analysis. A tool-augmented agent does not merely reason hypothetically about what a vulnerable application might do; it sends actual requests, receives actual responses, and reasons from actual target behaviour. This distinction is fundamental.

Schick et al. (2023) demonstrated that LLMs can learn, in a self-supervised manner, to determine when and how to use tools—not through explicit fine-tuning on tool-use datasets but through exposure to examples of productive and unproductive tool invocations [?]. This suggests that tool selection is a learnable reasoning skill rather than a hard-coded lookup, which has important implications for agents that must choose among diverse security testing toolkits depending on what the target reveals.

Qin et al. (2023) extended this finding to show that instruction-tuned LLMs can generalise tool-use capabilities to tool sets not represented in their fine-tuning data, relying on structured tool descriptions to infer correct usage patterns [?]. For security agents

deploying novel testing capabilities, this is directly relevant: the agent should be able to reason about how to use a new testing function from its description, without the need for fine-tuning on that specific function’s usage.

Despite these capabilities, tool augmentation introduces three structural costs that compound the problems already identified for ReAct:

Context pollution by tool output: Raw tool outputs—complete HTTP responses including headers and body, verbose nmap scan results, detailed directory listings—can consume hundreds to thousands of tokens per invocation. In a session of several hundred tool calls, the cumulative output consumes the overwhelming majority of context window capacity, accelerating the trajectory explosion problem and displacing the strategic reasoning traces that should inform subsequent action selection.

Selection noise in large tool spaces: As the number of available tools increases, the cognitive load on the agent for tool selection increases proportionally. Without a structured model of what has already been tried, the agent’s tool selection at each step may not reflect the accumulated evidence of what has and has not been productive. The same unstructured informational noise that causes the agent to re-test payloads can cause it to re-apply entire tool workflows to surfaces already exhausted.

Output parsing reliability: Translating the unstructured, format-variable output of security tools into structured inferences about target state requires language model generation—and that generation is subject to the hallucination tendencies documented by Ji et al. (2023) [?]. A model operating from a long, compressed context window may generate incorrect inferences about what a tool output means, and those incorrect inferences, once part of the trajectory, shape all subsequent reasoning without any mechanism for correction.

The Planning Versus Reactive Execution Tension

The tension between deliberative planning and reactive execution is one of the oldest and most deeply studied problems in AI agent design [?], and it manifests with particular acuity in security domains. Understanding it analytically is necessary for characterising why current LLM security agents occupy an unstable middle ground between both poles.

Deliberative planning approaches commit to a course of action before taking any steps, reasoning about the likely trajectory from current state to desired outcome. Their strength is strategic coherence: a plan provides principled ordering of actions that avoids redundancy and targets the highest-value opportunities first. Their weakness, as discussed in the CoT analysis, is brittleness to unexpected evidence that invalidates planning assumptions.

Reactive execution approaches select the next action based solely on the most recent observation, without maintaining any model of the overall task structure. Their strength is adaptability: no planning assumption can be violated because none was made. Their weakness is the absence of strategic direction: the agent tends toward short-sighted, locally responsive behaviour that may explore locally promising paths indefinitely while neglecting globally important surfaces.

ReAct’s position in this spectrum is to use verbal reasoning traces as a lightweight proxy for planning—generating a brief strategic rationale before each action, drawing on whatever history remains accessible in the context window. This is strictly better than pure reaction (the reasoning trace provides at least local strategic guidance) and strictly better than pure commitment (the reasoning can be revised at each step). But it inherits the fundamental weakness of both extremes: the strategic guidance is only as reliable as the context that informs it (which degrades over the session), and the adaptation is only as powerful as the reasoning traces are accurate (which degrades when the context is compressed).

No architecture in the published security agent literature has resolved this tension in a principled, session-length-independent way. It remains an open research problem.

3.4.3 Memory Mechanisms in LLM-Based Security Agents

Memory is not a peripheral concern in agent design; it is the central architectural property that determines whether a system can sustain coherent, non-redundant strategy across an extended engagement. Three broad approaches to memory have been applied in LLM-based systems, including security agents. Each has distinctive capabilities and structural limitations.

In-Context Trajectory Memory

In-context memory is the universal baseline: the agent’s entire memory of its session is the conversation trajectory accumulated in the context window. Its advantages are simplicity (no external infrastructure), completeness (every observation is nominally present), and seamless integration with LLM generation (the model processes the trajectory directly on each call).

Its limitations are the ones already identified in the ReAct analysis: context window overflow (later content displaces earlier content), attention dilution (middle-of-window content is under-attended), and the absence of any structured query mechanism (the agent cannot efficiently determine whether it has already tested a specific surface without reading through the entire history).

For penetration testing specifically, the absence of structured queries is not a convenience limitation—it is a fundamental capability gap. An expert tester can instantly answer questions like “have I tested IDOR on the user endpoint with role elevation?” or “which of the confirmed XSS findings has a stored rather than reflected variant?” These are structured queries over a record of testing history. An agent that must infer the answers from natural language history cannot replicate this capability reliably across sessions of realistic length.

Retrieval-Augmented Memory

Retrieval-Augmented Generation (RAG) [?] extends in-context memory by storing information in an external vector database and retrieving semantically relevant subsets for each generation step. This addresses the context overflow problem: the external store can hold arbitrarily much information, and only the most relevant subset is loaded into the context window for each step.

RAG has demonstrated genuine value on knowledge-intensive tasks where the retrieved information takes the form of documents or passages that are semantically related to the current query [?]. Its adaptation to security agent memory, however, encounters three structural problems. First, security agents need *structured* queries over their testing history (“what is the current status of hypothesis H7?”), not semantic similarity retrieval. Vector similarity search maps semantic relatedness, not structured record lookup; it will not reliably distinguish between “I tested this endpoint” and “I hypothesised about testing this endpoint.” Second, vector retrieval is context-agnostic: the similarity between a stored finding and the current query is computed without knowledge of the agent’s current strategic priorities, so retrieved content may be semantically relevant but strategically useless. Third, testing state is mutable in ways that static document retrieval cannot handle: a hypothesis transitions from open to testing to confirmed or rejected; these transitions update the record, and the retrieval system must reflect current status rather than historical assertions.

External Structured Memory

The MemGPT approach [?] addresses RAG’s structured-query limitation by enabling LLMs to explicitly manage their own external memory through read and write operations against a structured store. The agent reasons about what to store, retrieves specific information on request, and maintains structured records that persist outside the context window indefinitely.

This is architecturally closer to what penetration testing requires: structured, mutable,

queryable state that is not subject to context window limitations. The structural limitation is what might be called the *agent interface problem*: the agent’s reads and writes to the external store are themselves LLM-generated operations. If the model hallucinates an incorrect status for a hypothesis—recording a confirmed finding as rejected, or vice versa—the error is committed to the persistent store and shapes all subsequent reasoning. There is no independent verification mechanism that can catch this error before it propagates. The reliability of the structured memory is therefore bounded by the reliability of the LLM-generated write operations that maintain it, and under the long-context, degraded-attention conditions that prevail in a sustained security engagement, that reliability is precisely what cannot be guaranteed.

Table 3.2: Memory Mechanism Comparison for LLM Security Agents

Mechanism	Compressed- Session vival	Sur- port	Structured Query Sup- port	State tion Reliabil- ity	Muta- Reliabil- ity
In-context trajectory	None; attention dilution		None; implicit only	Not applicable (read-only)	
RAG (vector retrieval)	Full (external store)		Semantic similar- ity only	None; static doc- ument model	
External structured (MemGPT)	Full (external store)		Full explicit API	Partial; LLM- generated writes unverified	

3.4.4 Critical Analysis of Deployed LLM Security Agents

PentestGPT

PentestGPT [?] represents the most architecturally sophisticated LLM-based penetration testing system in the published literature at the time of this writing. Its key structural innovation is the separation of the monolithic agent into three specialised modules, each driven by a distinct prompt: a *reasoning module* responsible for high-level strategic decisions (what to test and why), a *generation module* responsible for translating strategic decisions into specific tool commands, and a *parsing module* responsible for interpreting tool outputs and distilling them into structured observations the reasoning module can act on.

The importance of this separation is evidenced by the performance gap between PentestGPT and an unaugmented LLM baseline. The same underlying model, when directly tasked with penetration testing without the specialised module structure, achieves

approximately 14.3% task completion on HackTheBox challenges. With the module separation, this rises to approximately 67%. The architecture, not the model, accounts for the majority of the performance gain. This is a direct empirical validation of the Architecture Primacy Hypothesis (H1) stated in Chapter ??.

The failure analysis is equally instructive. Deng et al. observe that the distribution of failed tasks does not concentrate on the technically hardest challenges [?]. Challenges requiring many sequential investigation steps to accumulate sufficient evidence for exploitation are disproportionately represented in the failure set, regardless of their nominal difficulty level. This is the empirical signature of the trajectory explosion problem: the agent’s difficulty is not reasoning about a complex vulnerability—it is maintaining strategic coherence across the many steps required to converge evidence sufficient for exploitation. The architecture successfully separates *types* of reasoning; it does not solve the *duration* problem.

HackingBuddyGPT

HackingBuddyGPT [?] provides a deliberately minimalist contrast to PentestGPT’s structured approach. A frontier LLM is given standard shell access and instructed to escalate privileges on a target Linux host, with no domain-specific structuring beyond a brief system prompt. The system’s performance demonstrates that LLM security reasoning is generalisable even without elaborate scaffolding: the model applies standard enumeration sequences, identifies SUID binaries and writable system files, and selects appropriate exploitation techniques from its pre-training knowledge rather than from provided templates.

The failure modes documented by Happe and Cito are analytically clear [?]. The most common pattern is command repetition: the agent reissues commands it has already issued and received results for, because it cannot reliably determine from the conversation history what it has and has not already tried. This is the direct empirical manifestation of the action deduplication gap. The agent is not incapable of the reasoning required; it is incapable of tracking its own action history reliably enough to exclude already-attempted approaches from its action selection. The second most common failure pattern is strategic incoherence after a long session: an agent that had been exploring a coherent privilege escalation path appears to lose the thread of that path, issuing commands that are unrelated to the established strategy, as if starting fresh from an earlier point in the session. This is the attention dilution problem observed directly.

AutoAttacker

AutoAttacker [?] is notable as the only system in the surveyed literature that explicitly addresses the memory problem as an architectural concern, rather than treating it as a model capability issue. The system introduces a periodic heuristic summarisation step: at defined intervals, the accumulated conversation history is compressed into a natural-language summary, and the summary replaces the raw history in the context window. This directly addresses the context overflow problem.

The analytical limitation of heuristic summarisation as a fix for the memory problem is that it treats the symptom (context overflow) rather than the cause (the absence of structured, persistent state external to the conversation). Compressing a technical conversation into a natural-language summary loses the specific structured facts that subsequent reasoning requires. A summary that records “multiple SQL injection attempts were made on the product search endpoint with mixed results” loses: which specific payloads were tried, what the precise HTTP responses were, whether the response patterns were consistent with a boolean-blind or time-based vulnerability, and what follow-up probes those patterns suggest. These are not peripheral details; they are the structured data from which the next steps in the investigation must be derived. AutoAttacker improves session-length scalability at the cost of structured fact preservation—a trade-off that is appropriate as an engineering workaround but is not a solution to the underlying architectural problem.

Table 3.3: LLM-Based Security Agents: Comparative Dimensional Analysis

System	Reasoning Archite- cture	Autonomy	Adaptability Constraint	Memory Strategy and Limita- tion
PentestGPT [?]	Module-separated (Reasoning / Generation / Parsing)	Semi-autonomous; human confirmation at checkpoints	Evidence-driven within each module; no cross-module persistent state	Disjoint per-module trajectories; confirmed findings not durably tracked across modules
HackingBuddyGPT [?]	Monolithic reactive loop; minimal scaffolding	Fully autonomous over session	Directly responsive to terminal output; no global strategy model	Uncompressed linear conversation history; loses coherence after ~100 steps
AutoAttacker [?]	CoT planning phase followed by reactive execution loop	Fully autonomous	Moderate; plan-reality divergence accepted; reactive phase adapts locally	Heuristic natural-language summarisation; compresses away specific structured facts

3.4.5 Cross-Cutting Analysis: What the Convergence of Failures Reveals

The most analytically significant observation across the three evaluated systems is the convergence of their primary failure modes despite their architectural diversity. PentestGPT uses module separation and careful prompt design. HackingBuddyGPT uses minimal scaffolding and raw model capability. AutoAttacker uses heuristic summarisation and planning phases. These are substantively different architectural approaches. Yet all three fail predominantly on tasks requiring sustained, structured awareness of what has been done and found over the course of a long session.

This convergence is not coincidental. It is a logical consequence of the one property all three systems share: the trajectory-as-memory model. In all three, the agent’s primary persistent state is stored in natural language conversation history. The architectural differences between the systems are differences in how that history is managed (compressed, modularised, or raw), not differences in its fundamental nature. As long as the trajectory remains the primary memory substrate, the consequences of its structural limitations—context overflow, attention dilution, absence of structured queries—will manifest in essentially all systems that share it, regardless of how the surface-level architecture differs.

Chapter 4 Research Gap Taxonomy

The comparative analysis in Chapter ?? and the causal synthesis in Chapter ?? jointly establish the evidential basis for a formal taxonomy of the field’s open research problems. Six gaps are identified. Each is stated clearly, justified by specific evidence from the analysis, and characterised in terms of why it cannot be closed by optimisation within existing paradigms.

4.1 Gap Taxonomy

G1 – Semantic Generalisation Deficit

No existing automated penetration testing system can reliably detect vulnerability classes whose identification requires reasoning about application intent—including business logic flaws, contextual access control failures, and semantically novel instances of known vulnerability types—without prior encoding of the specific vulnerability’s signature.

Evidence: Díaz-Verdejo et al.’s evaluation [?] confirmed near-zero recall on OWASP A01 (Broken Access Control) and A04 (Insecure Design) across all evaluated DAST tools. These categories represent the most prevalent real-world vulnerability classes and are structurally inaccessible to recognition-based tools by the analysis in Section 3.2. LLM-based agents partially address this gap—PentestGPT’s 67% success rate on semantically complex HackTheBox challenges establishes partial closure—but do not fully close it, as evidenced by the remaining 33% failure rate and the documented degradation on long-session tasks where maintained semantic context is required.

Why the gap persists: Full closure requires both the semantic reasoning capability that LLMs provide and the sustained, reliable access to structured contextual information that current architectures do not provide. The gap is compound in nature: addressing the reasoning component (the model can reason) without addressing the scaffolding component (the model cannot access its accumulated evidence reliably at scale) produces the partial performance demonstrated by current systems, not the full performance that expert human testers achieve.

G2 – Heuristic Brittleness and False Positive Inflation

Context-blind payload injection produces false positive rates of 30–60% on complex applications, requiring manual triage that negates the scalability benefit of automation and systematically erodes practitioner trust in automated findings.

Evidence: Díaz-Verdejo et al.’s evaluation reported false-positive rates in this range across evaluated DAST tools [?]. The causal mechanism is context blindness: uniform payload application across all parameters treats non-vulnerable input validation as indistinguishable from vulnerable processing—a consequence of having no model of each parameter’s intended purpose.

Why the gap persists: The false-positive problem is not addressable by refining payloads, because the distinction between a vulnerable and a non-vulnerable parameter is not observable from the parameter’s response to injection alone; it requires understanding the parameter’s intended purpose, which requires semantic reasoning about the application. LLM-based agents can, in principle, apply this semantic reasoning to winnow false positives; but the extent to which current LLM agents actually reduce false-positive rates relative to traditional DAST tools has not been systematically measured in the published literature.

G3 – Finding-Relationship Chain Blindness

No existing automated penetration testing system maintains a persistent, queryable representation of the relationships between confirmed findings that would enable systematic reasoning about multi-step vulnerability chains across the full duration of an engagement.

Evidence: LLM-based agents can perform chain reasoning within a single inference step—the model, given confirmation of an SSRF finding, can articulate what subsequent steps the finding enables. But this chain reasoning exists only in the moment of its generation. After compression, the chain context from 50 steps ago may be inaccessible at step 150, and the opportunity is missed. The failure to pursue confirmed exploitation chains, documented in PentestGPT’s long-session failures [?], is the empirical signature of this gap.

Why the gap persists: A persistent finding-relationship graph—one that survives context compression events and can be queried at any point in the session—is not implemented in any published security agent architecture. Its absence is a consequence of the trajectory-as-memory model, which stores all finding information in natural language conversation history rather than a queryable structured representation.

G4 – Stateless Reasoning and Context Degradation

Every LLM-based penetration testing system in the published literature relies on conversation trajectory as its primary persistent state. Confirmed findings, rejected hypotheses, and tested payloads are subject to irreversible loss through context compression and systematic under-attention, causing strategic degradation that worsens monotonically with session length.

Evidence: This gap is directly evidenced by the correlation between session length and failure rate in PentestGPT [?], the command-repetition failures in HackingBuddyGPT [?], and the explicit memory mitigation strategy (heuristic summarisation) deployed by AutoAttacker [?]. The causal mechanism is established in the synthesis: the information bottleneck of the context window is a fixed capacity meeting exponentially growing session content.

Why the gap persists: Addressing this gap requires externalising essential testing state to a store that is independent of the context window and immune to compression events. No current system achieves this for the specific structured state (hypothesis status, payload history, coverage tracking) that penetration testing requires. Larger context windows delay but do not eliminate the problem; heuristic summarisation trades overflow for fact loss.

G5 – Absent Hypothesis Lifecycle Management

No existing LLM-based security agent maintains a formal, structured registry of security hypotheses by lifecycle status (open, under investigation, confirmed, rejected) with associated payload deduplication, causing redundant retesting after compression events and preventing principled exclusion of exhausted surfaces from future action selection.

Evidence: The thrashing behaviour documented in both PentestGPT and HackingBuddyGPT evaluations—agents retesting surfaces they have already exhausted—is the direct empirical signature of this gap [?, ?]. Expert human testers manage their hypothesis space explicitly, either mentally or through notes; current automated agents do not.

Why the gap persists: Hypothesis lifecycle management requires a structured state representation in which the lifecycle status of each surface-vulnerability pair is maintained independently of the conversation history and is updatable reliably. This is the same architectural requirement as G4 (external structured state), applied specifically to the hypothesis management use case. The gap is therefore a special case of the broader stateless reasoning problem, but merits separate identification because its operational consequences (payload redundancy, failure to converge) are distinct from the general

strategic incoherence caused by G4.

G6 – Safety and Accountability Architecture Absence

No existing LLM-based penetration testing system provides safety enforcement adequate for responsible autonomous offensive operation: no formal resistance to prompt injection via target-embedded content, no verifiable scope enforcement, and no cryptographically secured audit trail.

Evidence: Prompt injection via adversarially crafted server responses is a documented and demonstrated attack vector against LLM-integrated applications [?]. A security agent that processes target HTTP responses directly as part of its reasoning context is structurally exposed to this attack: a malicious target can embed instruction-like content in its responses that, if processed without sanitisation, may redirect the agent’s planning toward actions outside its authorised scope. Current security agent systems address this risk through soft constraints communicated via system prompt conditioning—an approach that is definitionally bypassable by adversarial content specifically designed to override it.

Why the gap persists: Closing this gap requires safety enforcement mechanisms that are architecturally independent of the LLM’s reasoning layer: scope validators that check targets before network connections are established, prompt injection filters that sanitise tool outputs before they reach the context window, and audit logging maintained in a form the agent cannot modify. These are infrastructure concerns that the current literature treats as engineering add-ons rather than architectural requirements.

4.2 Gap Interactions and Compounding Effects

Table 4.1: Research Gap Taxonomy: Evidence Map and Interaction Matrix

Gap	Primary Evidence	Evidence	Paradigm Affected	Compounds With
G1 – Semantic Generalisation	Diaz-Verdejo [?]; Liu et al. [?]		All generations	G4
G2 – Heuristic Brittleness	Diaz-Verdejo (30–60% FPR) [?]		Gen 1; partially Gen 3	G1
G3 – Chain Blindness	PentestGPT failure analysis [?]		Gen 3 only	G4, G5
G4 – Stateless Reasoning	All Gen 3 systems; Liu et al. [?]		Gen 3 only	G3, G5, G6
G5 – Hypothesis Lifecycle	Thrashing in PentestGPT, HackingBuddyGPT [?]		Gen 3 only	G4
G6 – Safety Absence	Greshake et al. [?]		Gen 3 only	G4

The gaps are not independent. G3, G4, and G5 form a particularly destructive compounding cluster. An agent without persistent state (G4) cannot track hypothesis lifecycle status (G5), and without hypothesis lifecycle tracking, it cannot maintain the record of confirmed findings required to reason about exploitation chains (G3). The failure to discover multi-step vulnerability chains in long-session evaluations is not attributable to any single gap in isolation; it is the combined effect of all three. A system that addresses G4 alone, without also addressing G5 and G3, will be better at maintaining session continuity but will still fail to converge on chain exploitation because it lacks the structured hypothesis and finding representations needed to reason about chains at scale.

Similarly, G6 is not independent of G4 and G5. An agent that does not reliably know what it has done (G4, G5) cannot produce a meaningful audit trail, because the agent’s own record of its actions is unreliable. The accountability component of G6 depends on resolving the memory components of G4 and G5.

Chapter 5 Conceptual Perspective

The research gap taxonomy identifies what is missing. This chapter articulates the theoretical properties that future approaches must provide—not as architectural blueprints or engineering specifications, but as conceptual requirements derived by logical inversion of the identified gaps.

5.1 The External State Principle

The trajectory-as-memory model is the common root cause of G3, G4, and G5. Inverting this root cause yields a theoretical principle: *for any autonomous agent operating over a decision horizon that exceeds what a single context window can reliably represent, the decision-relevant state of the ongoing investigation must be maintained in a structured store that is independent of the language model’s context, immune to compression events, and accessible through structured queries.*

This principle implies a specific partition of responsibility between the LLM and the memory system. The LLM’s responsibility is reasoning: given a compact, structured digest of the current state of the investigation, produce the next reasoning step and action selection. The memory system’s responsibility is state: maintain the ground truth of what has been tested, what has been found, and what the current strategic priorities are—not as an inference from natural language history, but as a directly maintained structured record.

This is a different model from both pure in-context memory and from RAG. In pure in-context memory, the LLM is responsible for both state maintenance and reasoning, and the state is entangled with the reasoning in natural language history. In RAG, retrieval is semantic rather than structured, and the retrieved information does not reflect mutable state. The External State Principle requires that the memory system explicitly manage state transitions—when a hypothesis is confirmed, the memory system records the transition; when an endpoint is tested with a specific payload, the memory system records the action—so that the LLM is never responsible for inferring state from history.

The consequence for system design directions is clear: priority should be placed not on context window size (which delays but does not eliminate the trajectory explosion problem) but on externalising the structured facts that penetration testing’s decision logic most critically depends on.

5.2 The Verified Evidence Principle

The unverified inference propagation mechanism identified in Chapter ?? suggests a second theoretical principle: *the confirmation of a security finding must require observable, independently verifiable exploitation evidence, not merely an LLM’s generated assertion of confirmation.*

This principle addresses the hallucination risk by introducing an architectural demarcation: a hypothesis transitions from “under investigation” to “confirmed” only when a verification layer external to the LLM generation process has examined the evidence and determined that it meets a defined confirmation standard. For a SQL injection hypothesis, confirmation requires that specific data has been extracted from the database by a query that the agent constructed—not that the agent’s reasoning trace asserts that injection was successful. For an access control hypothesis, confirmation requires that a response was received containing data from another user’s account—not that the agent inferred this was probably the case.

This verification boundary serves two distinct purposes. First, it prevents hallucinated confirmations from propagating into subsequent strategic reasoning, because the external verifier catches false assertions before they are recorded as facts. Second, it provides the empirical basis for meaningful reporting: a finding reported through a verified-evidence architecture can be demonstrated to a stakeholder with the specific, reproducible exploitation evidence that confirmed it, not merely with a reasoning trace asserting its probability.

5.3 The Hierarchical Decomposition Principle

The planning-versus-reaction tension identified in the ReAct analysis, and the chain blindness of single-agent loops (G3), jointly suggest a third theoretical orientation: *the exploration-exploitation and breadth-depth trade-offs that single sequential agent loops cannot resolve optimally may be addressable through hierarchical agent decomposition with shared external state.*

In a hierarchical architecture, a controlling agent maintains strategic oversight—managing the external state representation, identifying the most valuable next investigation direction based on the current finding graph, and delegating specific investigation tasks to specialised sub-agents. Each sub-agent operates with a constrained, well-defined context (a specific endpoint, a specific vulnerability class, a specific exploitation chain) and terminates when its task is complete, returning its findings to the external state for the controlling agent to incorporate. Because sub-agents operate concurrently, the exploration-exploitation tension is resolved by allocation rather than scheduling: one

sub-agent explores new surfaces while another exploits a confirmed finding.

The critical enabling condition for this architecture is the external state store required by the External State Principle: without a shared, persistent, structured record of findings, concurrent sub-agents would either duplicate effort or miss interaction effects between their respective investigations. The hierarchy resolves the strategic coordination problem; the external state enables the hierarchy to function without constant full-context communication between agents.

5.4 Conceptual Orientation of the Phantom Research Project

The PHANTOM project, which contextualises this thesis, constitutes a research-oriented exploration of the three theoretical principles articulated above in a specific domain. Without characterising its implementation, its conceptual positioning can be described at the theoretical level: it seeks to operationalise the External State Principle through domain-specific persistent state structures maintained outside the LLM context window; it explores the Verified Evidence Principle through confirmation mechanisms that require observable exploitation evidence before recording a finding as confirmed; and it explores the Hierarchical Decomposition Principle through concurrent sub-agent delegation with shared persistent state access.

PHANTOM is offered not as a solution to the research gaps—the gaps are not resolved by any current system—but as a conceptual exploration of the theoretical principles suggested by the gap analysis. Its value for this thesis is that it provides concrete grounding for the principles: implementing them surfaces the specific engineering constraints and design trade-offs that purely theoretical analysis does not reveal, and those revealed constraints refine the theoretical understanding of what the principles require.

Chapter 6 Discussion

6.1 Implications for Cybersecurity Practice and Governance

The analysis conducted in this thesis has implications that extend beyond the academic research agenda to the practice and governance of cybersecurity assessment.

The automation complacency risk. Organisations that deploy automated scanning tools as their primary security assessment mechanism risk a false sense of coverage. The analysis establishes that first-generation tools cannot detect the most prevalent and impactful vulnerability classes without expert human reasoning, and that second- and third-generation tools, while more capable on specific task types, degrade in ways that are not visible to an organisation monitoring output volume rather than finding quality. An organisation that concludes “our scanner found nothing critical” may be drawing a conclusion from a tool that is structurally incapable of finding what it is looking for. Governance frameworks for automated security assessment should explicitly account for the semantic generalisation deficit (G1) and the false positive inflation problem (G2) in their interpretation of automated tool outputs.

The LLM over-optimism problem. Academic results demonstrating frontier LLM security reasoning capability are frequently communicated in the practitioner press in ways that overstate their generalisability. Liu et al.’s 87% CVE exploitation rate [?] is an extraordinary demonstration of capability on a specific task (exploiting vulnerabilities given structured descriptions) that does not directly translate to a general penetration testing capability. The same study’s 7% success rate without structured context is a more accurate indicator of what an LLM-based system achieves without the architectural scaffolding that organisations deploying these systems may not have. Practitioners and security managers should understand the distinction between demonstrated short-horizon capability and the as-yet-undemonstrated sustained engagement capability that real-world assessments require.

The human-AI collaboration opportunity. The analysis does not support a pessimistic conclusion about LLM-based security agents. Rather, it suggests a more nuanced positioning: these systems are genuinely capable contributors to the reconnaissance and initial surface-coverage phases of a penetration testing engagement, where their semantic generalisation over known vulnerability patterns provides measurable value

over traditional scanning. The phases where they currently fail—sustained multi-step chain exploitation, long-horizon strategic coherence, reliable finding confirmation—are precisely the phases where expert human judgment provides the most irreplaceable value. An architecture in which AI agents provide comprehensive surface coverage and human practitioners focus their effort on the deep-reasoning exploitation steps that the AI has prepared is both more scalable than purely human assessment and more reliable than purely autonomous AI assessment. The research community should focus on developing the architectural foundations that make this collaboration tractable.

6.2 Limitations of This Analysis

The critical analysis in this thesis rests on a body of published empirical research that is itself subject to limitations that constrain the strength of conclusions.

Benchmark heterogeneity. The systems analysed in Chapter ?? were evaluated on different targets, with different session budgets, against different vulnerability types, using different LLM backends. PentestGPT was evaluated on HackTheBox challenges; HackingBuddyGPT on custom Linux privilege escalation hosts; AutoAttacker on an undisclosed web application set. Direct quantitative comparison across systems is therefore not methodologically valid. The comparative claims in this thesis are qualitative and structural, based on failure-mode patterns rather than numeric performance rankings.

Intentionally vulnerable target bias. All published empirical results for LLM-based security agents involve intentionally vulnerable training applications (HackTheBox, DVWA, WebGoat) or purpose-built laboratory hosts. These differ from production applications in ways that are operationally significant: WAF protection, rate limiting, real-user traffic noise, and proprietary authentication mechanisms create conditions that fundamentally change both the information available to the agent and the consequences of its actions. The gap between laboratory results and production performance is unknown and likely substantial.

Rapid model capability evolution. The LLM capabilities characterised in the empirical literature are those of models available during the evaluation periods of 2022–2024. The models currently available are substantially more capable on many benchmarks. However, the structural limitations identified in this analysis—context window finiteness, attention non-uniformity, and the absence of external structured state—are architectural properties that are not eliminated by model capability improvements. The architecture primacy argument is model-capacity-independent.

6.3 Open Challenges and Research Risks

The benchmark design imperative. The absence of a standardised penetration testing benchmark is the field’s most consequential methodological gap. Without a shared evaluation protocol—common targets, known ground truth, defined iteration budgets, standardised confirmation criteria—published performance claims cannot be compared, verified, or aggregated into cumulative scientific progress. The research gap taxonomy G1–G6 established in Chapter ?? provides a natural framework for such a benchmark: a target corpus with at least one verified instance of each gap category, evaluated under controlled and reproducible conditions.

The dual-use tension. Advancing autonomous penetration testing capability directly advances autonomous attack capability. This is not a hypothetical risk; it is structural. Every architectural improvement that makes an agent more effective at sustained, multi-step exploitation under real-world conditions also makes it more effective as an adversarial tool when deployed without authorisation. The research community operating in this space has a responsibility to engage seriously with responsible disclosure norms, access control frameworks, and deployment governance—not as afterthoughts, but as first-class research concerns alongside the technical advances themselves.

The formal verification challenge. The safety properties required for responsible autonomous offensive operation—verifiable scope enforcement, prompt injection resistance, cryptographically auditable action records—are not adequately addressed by empirical testing alone. A system that passes a large set of adversarial test cases remains exposed to attacks not represented in the test set. Formal verification methods—model checking for scope enforcement logic, verified isolation for tool execution environments, cryptographic proof for audit trail integrity—are the appropriate standard for production-grade safety claims. Applying such methods to autonomous security agents is an open research problem that the current literature has not substantively addressed.

Chapter 7 Conclusion

7.1 Summary of Findings

This thesis has conducted a rigorous critical analysis of automated penetration testing across three generations of approach, directed toward two goals: understanding the precise structural causes of the persistent performance gap between automated systems and expert human assessment, and establishing a justified taxonomy of the specific research problems that must be solved to close that gap.

The first generation of automated tools—signature-based scanners, heuristic active testers, and domain-specific exploitation frameworks—demonstrated that automation could deliver fast, scalable, and reproducible detection of pre-catalogued vulnerability patterns, while demonstrating equally that this capability is categorically bounded by the template ceiling. The detection paradigm, by its nature, cannot generalise beyond what it has been taught to recognise.

The second generation extended this with rule-based orchestration that improved coordination and introduced primitive conditional adaptability, but remained bounded by the coverage completeness assumption: the system can only respond to situations that a human developer anticipated and encoded in advance.

The third generation—LLM-based autonomous agents—represents a genuine qualitative advance. These systems can reason about novel vulnerability instances without templates, adapt strategy based on evidence, and engage with the semantic complexity of real applications. The empirical record confirms this: PentestGPT’s 67% completion rate on expert-level challenges, HackingBuddyGPT’s autonomous privilege escalation from first principles, and Liu et al.’s 87% CVE exploitation with structured context all establish that the template ceiling has been breached.

The critical finding of this analysis is that breaching the template ceiling is not equivalent to achieving expert human performance, and the gap between the two is not a model capability gap that better LLMs will close. It is a structural architectural gap rooted in three causal mechanisms: the information bottleneck of fixed-capacity context windows, the implicit state representation embedded in natural language conversation history, and the unverified inference propagation that occurs when the same model is responsible for both reasoning and memory management without independent verification.

7.2 Evaluation of Research Hypotheses

H1 – The Architecture Primacy Hypothesis is supported by the evidence. Liu et al.’s 12.4-fold performance difference between structured-context and unstructured-context conditions [?], and the 4.7-fold performance difference between PentestGPT and its unaugmented baseline [?], both demonstrate that architecture is the dominant performance determinant for a given model. The binding constraint on current systems is not the model’s intrinsic security reasoning capability but the structural properties of the agent architecture surrounding it.

H2 – The Convergent Failure Hypothesis is supported by the comparative analysis. Despite the architectural diversity of PentestGPT, HackingBuddyGPT, and AutoAttacker, their primary documented failure modes converge on the same structural cause: the inability to maintain reliable, structured awareness of investigation state across sessions longer than the effective range of their context window. The convergence is not coincidental; it is logically derivable from the one property all three systems share.

7.3 The Research Gap Taxonomy: A Standing Contribution

The six-item research gap taxonomy established in Chapter ??—Semantic Generalisation Deficit (G1), Heuristic Brittleness (G2), Chain Blindness (G3), Stateless Reasoning (G4), Absent Hypothesis Lifecycle (G5), and Safety Architecture Absence (G6)—constitutes the primary scientific contribution of this thesis. Each gap is:

- **Justified** by specific empirical evidence from the published literature or by logical derivation from documented architectural properties.
- **Architectural in character**: not closeable by quantitative improvements within existing paradigms, but requiring category-level redesign.
- **Appropriately scoped**: specific enough to guide research design while general enough to apply across architectural variants.
- **Interactionally characterised**: G3, G4, and G5 are identified as forming a compounding cluster whose combined effect exceeds the sum of individual gaps.

7.4 Research Agenda

From the gap taxonomy and conceptual perspectives, four research directions emerge as highest priority:

Domain-specific external state architectures: Designing and evaluating memory architectures for penetration testing agents that maintain hypothesis lifecycle status, payload history, and finding-relationship graphs external to the LLM context window, with automatic (not LLM-managed) state transitions triggered by agent actions. The appropriate evaluation metric is not just finding recall but the rate at which strategic coherence degrades as session length increases.

Verified confirmation standards: Establishing community-agreed standards for what constitutes confirmation of each vulnerability class in autonomous security testing, implementing verification layers that enforce these standards independently of LLM reasoning, and measuring how verified confirmation rates compare to unverified claim rates across current systems.

Standardised penetration testing benchmarks: Designing a shared evaluation framework structured around the G1–G6 taxonomy, with verified ground truth across all six gap dimensions, controlled session budgets, and reproducible evaluation protocols. This is a prerequisite for scientific accumulation in the field.

Safety-first architecture research: Developing and formally verifying safety mechanisms—prompt injection sanitisation, scope enforcement, audit trail integrity—that are independent of LLM reasoning and provide provable guarantees rather than empirical assurance. This is the precondition for responsible advancement toward production-grade autonomous penetration testing, and it must be treated as a research problem coequal with capability advancement, not as an engineering add-on applied after capability goals are achieved.

7.5 Closing Remarks

The distance between the first demonstration that an LLM can autonomously exploit a security vulnerability and the practical deployment of a reliable, safe, expert-level autonomous penetration testing system is substantial. This thesis’s analytical contribution is to characterise that distance precisely—not in terms of raw model capabilities, but in terms of the specific architectural properties that current systems lack, the specific task-structure demands that current architectures cannot satisfy, and the specific research directions that close the gap between what has been demonstrated and what is needed.

Penetration testing automation has been a stated goal of the security community for as long as automated tools have existed. The first generation made it fast and scalable within

narrow bounds. The third generation made it semantically capable within short time horizons. The next generation must make it structurally coherent across the full duration of a real engagement. That transition requires the field to shift its primary research focus from model capability to agent architecture—from asking “how smart is the model?” to asking “does the surrounding system give the model what it needs to be effective?” The research gap taxonomy established in this thesis is offered as a starting point for that shift.

Bibliography

- [1] Bishop, M. (2003). *Computer Security: Art and Science*. Addison-Wesley Professional.
- [2] Deng, G., Liu, Y., Mayoral-Vilches, L., et al. (2023). PentestGPT: An LLM-empowered automatic penetration testing tool. *arXiv preprint arXiv:2308.06782*.
- [3] Díaz-Verdejo, J., Málaga, M., Pereñíguez, F., and García-Teodoro, P. (2022). On the evaluation of dynamic analysis tools for security assessment of web applications. *IEEE Access*, 10:64874–64887.
- [4] Greshake, K. et al. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.
- [5] Happe, A. and Cito, J. (2023). Getting pwned by AI: Penetration testing with large language models. In *Proc. ESEC/FSE 2023*, pp. 2082–2086. ACM.
- [6] IBM Security (2024). *Cost of a Data Breach Report 2024*. IBM Corporation. <https://www.ibm.com/reports/data-breach>.
- [7] ISC2 (2023). *Cybersecurity Workforce Study 2023*. <https://www.isc2.org/research>.
- [8] Ji, Z. et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- [9] Kennedy, D., O’Gorman, J., Kearns, D., and Aharoni, M. (2011). *Metasploit: The Penetration Tester’s Guide*. No Starch Press.
- [10] Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS 2020*.
- [11] Liu, N. F. et al. (2023). Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*.
- [12] Liu, R., Yang, Z., Li, C., and Yang, H. (2024). Evaluating LLMs for privilege-escalation scenarios. In *Proc. 33rd USENIX Security Symposium*.
- [13] ProjectDiscovery (2020). *Nuclei: Fast and customisable vulnerability scanner*. <https://github.com/projectdiscovery/nuclei>.

- [14] OWASP Foundation (2021). *OWASP Top 10 — 2021*. <https://owasp.org/Top10/>.
- [15] OWASP Foundation (2010–present). *OWASP Zed Attack Proxy (ZAP)*. <https://www.zaproxy.org/>.
- [16] Packer, C. et al. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- [17] PTES Technical Guidelines (2012). *The Penetration Testing Execution Standard*. <http://www.pentest-standard.org/>.
- [18] Qin, Y. et al. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.
- [19] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
- [20] Schick, T. et al. (2023). Toolformer: Language models can teach themselves to use tools. In *Proc. NeurIPS 2023*.
- [21] Stampar, M. and the SQLMap project (2006–present). *SQLMap: Automatic SQL Injection and Database Takeover Tool*. <https://sqlmap.org/>.
- [22] Vaswani, A. et al. (2017). Attention is all you need. In *Proc. NeurIPS 2017*.
- [23] Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proc. NeurIPS 2022*.
- [24] Xu, X. et al. (2024). AutoAttacker: A large language model guided system to implement automatic cyber-attacks. *arXiv preprint arXiv:2403.01038*.
- [25] Yao, S. et al. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- [26] Yao, S. et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS 2023*.